

Robotic Autonomous Cameraperson Research-Based Project

Software Engineering Senior Project
Rochester Institute of Technology

Technical Report

Team: Ryan Yocum, J Russ, Kai Frazier, Connor O'Neill, Hiroto Takeuchi

Project Sponsor: Samuel Malachowsky

Faculty Coach: Larry Kiser

Table of Contents

Table of Contents	2
Project Overview	3
Basic Requirements	3
Constraints	4
Development Process	5
Project Schedule: Planned and Actual	5
System Design	6
Overall System Architecture	6
Commander	7
Operator	9
ICD Packet Structure	9
Socket	10
Sub	10
Arm	10
Scorbot	11
Ichor	11
ESP Driver	12
Menuconfig	13
I2C Bus	13
MotorHAT	13
ADS1015	13
Driver Wifi/Driver Socket	14
Encoder	14
Limit Switch	14
Hardware	15
Process and Product Metrics	16
Measurement Strategy	16
Quantitative Results	16
Performance Analysis	16
Product State at Time of Delivery	16
State of Product and Missing Features	16
Unplanned Implemented Features	17
Discrepancy	17
Project Reflection	18
Right	18
Wrong	18
Proposed Changes	19
References	19

Project Overview

This research project seeks to repurpose the ScorBot ER-4PC and ER-V, an educational robotic arm, into an autonomous robotic cameraperson. While basic videography techniques are well understood, capturing high-quality footage in dynamic environments like classrooms or live presentations requires constant operator attention. The project's goal is to automate camera operations—such as panning, tilting, zooming, and framing subjects—to create more engaging recordings. This project includes the ideation, research, and development of software that integrates machine learning, computer vision, and other technologies to guide camera movements.

The previous iteration of this project completed the full control of the ER-V with a basic tracking algorithm and a basic driver for starting to control the ER-4PC without its original controller. Our scope plans to further the research by completing the functionality of the ER-4PC and refactoring the controller to interface with both robots simultaneously. We will also be modifying the algorithm to use better framing and videography techniques, as well as working with both camera angles to swap between. Additionally, we plan to refactor existing code in order to increase the maintainability of the system.

Basic Requirements

The system must allow the user to connect robots to the Commander application via Operator or ESP Driver, and connect the robots' mounted webcams to Commander as well via the Camera-streamer program running on a Raspberry Pi. Once this setup is complete, the system must allow the user to disconnect and reconnect with robots without re-entering the robots' network information. They must also be able to switch between manual and automatic control of each robot. When in manual mode, the user should be able to control both axes of movement of the robot and trigger a homing sequence. When in automatic mode, the robot must autonomously track a person detected in the camera frame. The user should be able to select which object detection model is used for tracking, and have the option to view the bounding boxes generated by the model. The video stream used for tracking must also be exportable in a form that allows the user to choose what to do with it (for example, stream it to Zoom, save it to a file, etc.).

System Inputs:

- Robot network connection information: manually transferred from the robot control program (Operator or ESP Driver) to Commander.
- Manual movement commands: input by the user when connected to a robot in manual mode.
- Camera stream: captured by the webcams mounted on the robot arms.

System Outputs:

- Video stream: the video stream captured by the camera is sent to a virtual camera, from where the user can control where it goes
- Performance metrics: accessible in the TUI version of Commander via a pop-up window

Human Operator Characteristics:

- The average user of this system does not possess in-depth technical knowledge but rather a basic understanding of networking. This means that the setup of the system must be kept as simple as possible.

Safety Requirements:

- A hardware emergency stop button for killing power to the robot in potentially dangerous situations.

Constraints

- **Time:** Since we were limited to two semesters of development time, we had to prioritize heavily what we worked on, and be careful not to add unnecessary risk that would result in the project being unmaintainable for future development.
- **Money:** The hardware development lacked professional knowledge. Due to this, breaking parts was more common. Breaking things meant having to purchase new parts. Our R&D costs were limited, as well as by the goal of making this project accessible to hobbyists, which meant we couldn't use expensive parts. These factors required us to keep component costs to a minimum.
- **Accessibility:** Building this project for hobbyists and the open-source community means keeping it usable to people with minimal technical knowledge and skills. This meant that the robots themselves had to remain completely original, and we had to be careful choosing hardware that did not conflict with others being able to use our system.

Development Process

Our original development process we chose was based on the Spiral methodology, which was defined as 4 phases, each with 4 subsections: Planning, Research & Design, Engineering & Testing, and Evaluation. After the halfway point in the semester, we realized that this methodology was not working for our use case. The cards on the project board were not moving, and there was no real feature set being committed to. Afterwards, we decided to move to a milestone-based methodology, which was later realized to be a simplified SCRUM. We had planning meetings at the beginning of a milestone, where we planned features that we were shooting for to be completed at the end. These features were split into 3 different teams: Commander, Operator/ESP Driver, and Hardware. Although not all features were finished at the end of a milestone, the cards in the board were created, and allowed us during our meetings to glance over what was in progress.

The milestones aided our sponsor communication, as we scheduled demo meetings at the end of each milestone. Milestones were a month or two in length, so they demonstrated a larger feature set than a typical SCRUM sprint. Due to this being a research-based project with a vague scope from the sponsor, this process was defined by the team as it worked for the type of project and allowed us to collaborate in an efficient manner.

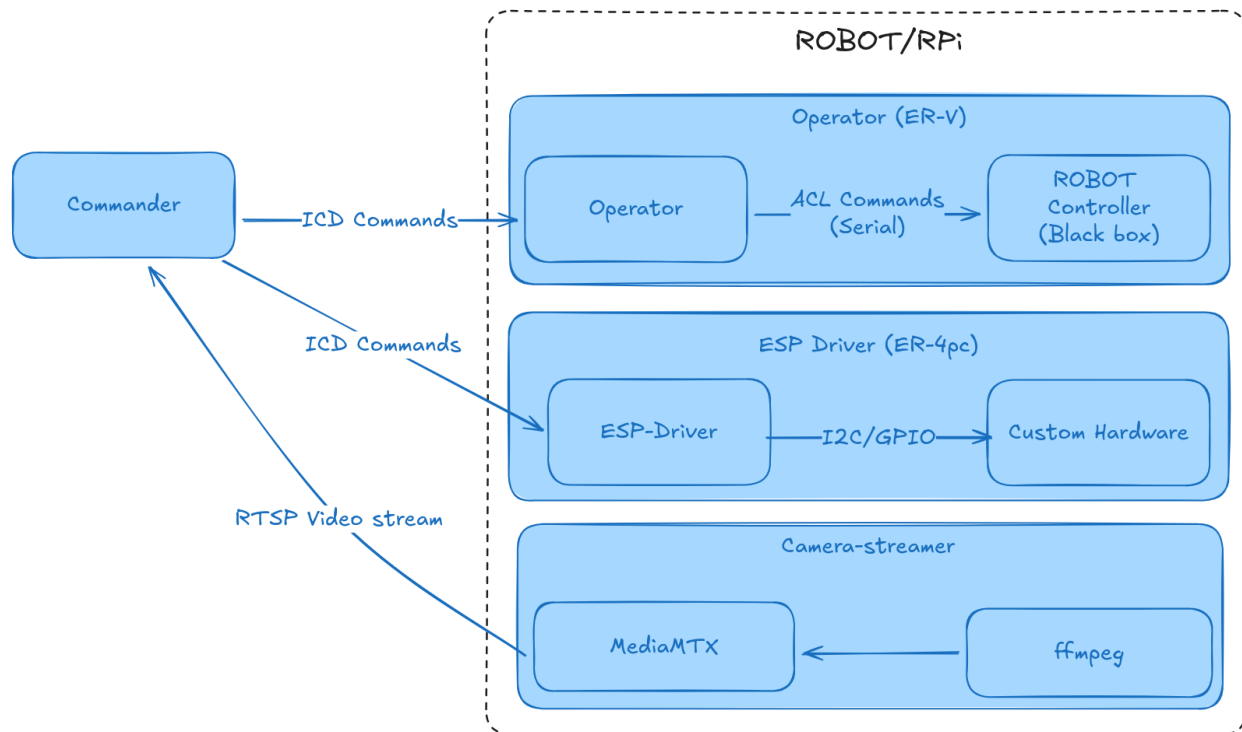
Project Schedule: Planned and Actual

For the first semester, we used Makerfaire, which takes place in November before Thanksgiving, as a deadline for our first major milestone. The reason for this was that we switched development methodologies from Spiral roughly a month into the semester. For this first milestone, our major goal was to get the hardware for manual motor control of the ER-4PC working and to be able to connect to multiple robots using Commander. Our next milestone was what we called a half-milestone, and its deadline was the end of the semester. We used this for creating some documentation based on what we had done and what we had learned from MakerFaire. We also roughly planned our goals for the second semester.

We used Imagine RIT, which is roughly at the end of April, as the end-of-semester deadline for the second semester, and we also created mid-semester deadlines either before or after Spring break in March to make a mid-semester deliverable and demo. During milestone 2, which was scheduled to end before spring break, we set out to complete our drivers for the Raspberry Pi, make performance improvements to Commander, and add the ability to stream video out of Commander. During development of the Raspberry Pi drivers, we discovered that the Pi would not be fast enough for our use case, so we had to pivot to using an ESP32. This marked a significant change to our architecture and our scope, leading us to only target moving a few axes and maybe homing if we had time. We decided to mainly focus on this task during this milestone. Milestone 3, which ended at Imagine RIT, focused on wrapping up the project by presenting at Imagine and completing documentation, as well as all other required deliverables. Overall, we were generally pretty good at sticking to our deadlines for each milestone; however, one major hurdle for scheduling demos on time was the availability of the project stakeholders.

System Design

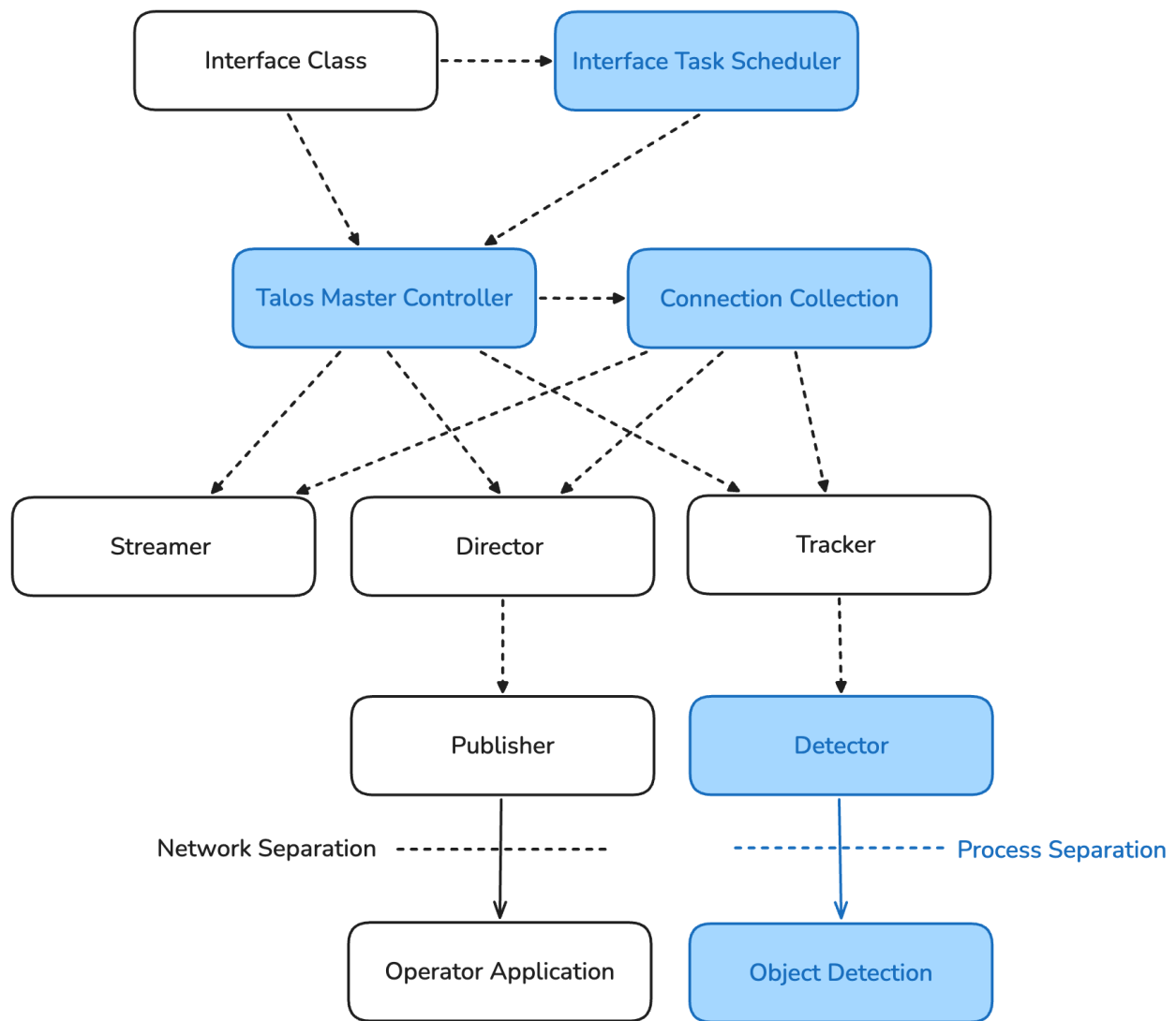
Overall System Architecture



The overall architecture of this system involves the Commander interface, which, at a high level, takes a camera feed and converts it into ICD commands, the Operator, which is currently only for the ER-V, interprets the ICD commands and turns them into ACL commands that the robot controller understands. The ESP Driver, which currently only works for the custom hardware on the ER-4PC, takes the ICD commands and converts them into motor movements, and then the Camera-streamer, which runs on the Pi and outputs an RTSP stream. This allows Commander to be run while not directly plugged into the webcam.

Most of this architecture was dictated from the previous iteration of this project, and it made sense, so we decided to keep it. The binary commands were already defined previously with good documentation on the format. ESP Driver was the largest change, and this was because we had to switch from a Raspberry Pi 4 to an ESP32. This was due to the hardware limitations of the Raspberry Pi to read encoders. The Camera-streamer was also added during this iteration to allow the Commander application to run anywhere without having to be plugged into the webcam. There is some noticeable delay when using the streamer, but this could be improved by switching to a simple IP camera that outputs an RTSP stream.

Commander



One of the major efforts we focused on in Commander is re-architecting the application to support new feature expansion and massive performance optimization.

In the original Commander from the previous iteration, we saw that the tracker class was handling the majority of the control logic for the application and was responsible for more than half of the features of the application. This proved to be a problem at the beginning as we constantly needed to update the tracker class and could not develop simultaneously with multiple developers without having large merge conflicts. The original commander also had several performance problems, where the user interface stuttered, and the object detection model was running at around 1 frame per second.

The solution we deployed was a major refactoring effort on Commander, including reorganizing logics and rearchitecting the class structure. For the new architecture, the first thing

we changed was to create a facade class that acts as a master controller, which abstracted the direct coupling of the tracker class from the user interface classes. This established a single, simplified point of contact for all user interface classes. This also proved to be effective as the Commander now has three completely different user interfaces, allowing us to easily extend the application for any interface library as we like. The Tracker class's logic has also been reduced and broken up into several controller classes, each with a single responsibility. Additionally, we added a new ConnectionCollection class that holds the model class for each robot connection. This was required as all controller classes needed access to the model classes. Having a single class for the entire application's state made the architecture significantly cleaner and allowed us to move logic around without having to worry about merge conflicts. This also completely decoupled all of the controller classes from each other and ran in isolation.

Another solution we added to Commander was proper task management. To reduce the workload on the main process, we created a new child process for object detection, allowing the detection process to run any size model without having to worry about performance affecting other sections of the application. We also added an additional class for scheduling new tasks. This class was made to allow dependency injection for a specific interface that defines its own way of scheduling workloads. While this was found to be somewhat effective, additional testing revealed that a generic thread-based scheduler works well with any interface, so we have defaulted to use that instead. However, we have kept this enhancement to allow for future implementations where the native interface scheduler has better performance.

Along with other minor refactoring work and deletion of unused code, we were able to significantly improve the model performance. So much so that the interface libraries' performance started to become a bottleneck for some devices, which is a very good problem to have. While this is all we have gotten with Commander and its performance enhancements this year, we recommend future teams to look into separating the interface of Commander to a web-based interface to reduce streaming workloads and improve support for a variety of OS.

Tangent to refactoring efforts, we have also begun laying out the foundation to stream out video and improve video selection for post-processing. While discussing the next feature sets within the team, we returned to our original domain model. From the domain model, we identified one section of the diagram that was completely missing in Commander, which was the post-production feature. From the refactoring above we were able to extract from Tracker class a brand new controller class called the Streamer. While this area still requires further refinement, we were able to implement two functional implementations of the streaming capability in Commander: the PyVirtualCam integration, which is capable of creating a virtual camera via the OBS software, and the FFmpeg streamer class capable of streaming raw RTSP video streams to integrate with browsers and other video software. We believe this area could be the next team's goal to extend, as it will be the foundation for features such as integrating new saliency-based detection models. This allows us to not only focus on streaming important videos but also pick frames that have the most sophisticated features.

Lastly, while no major changes were made, we have the Director class, which retained most of its logic of comparing detection results and commanding robots to move in selected directions. We also believe this branch of the architecture can be further improved with simulation and potential embodied large language models or vision language action (VLA)

models to further refine control schemes of the robots. It is also possible that this area could be reimplemented to support other API or network schemes, such as gRPC, something we considered implementing but deprioritized under ESP-32 driver code rewrite.

Regardless of what the future teams decide to focus on, we are confident that the new Commander architecture at the core provides a robust and easily extensible foundation, a legacy designed to significantly accelerate the work of all future teams.

Operator



The operator project was, for the most part, unchanged from the previous iteration of the project. Its architecture is designed as a one-way flow of data. It reads data off the TCP socket, uses a cyclic buffer to queue the command for processing, processes the command, and sends the ACL command over the robot's serial interface.

There are 3 threads active in this application. The first is the socket thread, which takes raw buffers and puts them into the Sub queue. The second is the Arm thread, which runs a loop pulling commands from the buffer, as well as sending serial commands to the robot. The third is the logger, which runs as a new thread independently to be out of the way of the important control threads.

ICD Packet Structure

Arg	C Type	Description
Message ID	UINT32	Unique ID for individual messages
RESERVED	UINT16	RESERVED
Command ID	UINT16	Command for device to carry out
Length	UINT16	Length of Payload
Payload	UINT8[]	Command Info
CRC	UINT8	XOR Checksum

This packet structure was designed to be a variable-length payload, with the length argument specifying the full length of the payload, and the Command ID to determine what type the payload is. The Message ID helps stop duplicate requests to prevent double processing identical requests, and the CRC is an XOR checksum to validate that the payload sent is

accurate. Due to network byte order, all of these packets are in big endian, so the data is endian converted from big to host byte order to be processed. The struct used to unpack these does not contain the CRC, and that is due to the payload being of variable length. The struct is unpacked manually using the pointer given from the payload field, and read until the length.

Socket

This module opens a plain TCP socket using `sys/socket.h`. It is set up as a server and set up to receive plain bytes, and the header is decoded using the ICD API described in the previous section. Each packet is pulled from the socket and put into a raw buffer in the Sub module.

Sub

Sub, short for Subscriber, is a module that acts as a queue, mostly. It is not a true subscriber/observer pattern, though. It is mainly a custom queue. The way that it is structured, there is one pool of buffers used, which are fixed and recycled. Each buffer in the pool can belong to one of the following

- *Free*
- *Command*
- *Return*
- *Log*

Only *Free* and *Command* are currently used. *Free* is the default for the pool. When a socket needs to put a command in the queue, it takes a buffer from the *Free* queue, adds the data, and puts it back into the *Command* queue. Then the Arm class removes from the *Command* queue, processes it, and returns it to the *Free* queue to be recycled for the next Socket command.

The reason for this module, which we inherited, was to facilitate communication between 2 different threads effectively. This method of a static-size pool prevents overflowing buffers, though it does come with the side effect that if commands come too quickly, they are just dropped. In important scenarios, this could include commands from Commander that would stop the robot. We decided to leave this as other protections should stop the robot in the event of that nature.

Arm

Arm is an abstract class that has a few concrete methods. One is `runLoop`, which runs the code for the arm, and the other is `processCommand`. `runLoop` is the main loop for the Arm, and calls `processCommand` every loop to pull commands from the queue. The individual movement commands, such as `polarPanStart` are defined in the subclasses, `Ichor` and `Scorbot`. This runs as its own thread and is responsible for all of the control of the robot.

Scorbot

This is a subclass of `Arm`, which defines the communication logic for the original controller over a serial port. It contains a sub-module, `ac1`, which converts the commands to the strings that the robot accepts. The robot is controlled in the robot's manual mode by holding down a key over serial to constantly be able to react. Commands are pushed to a command buffer and are dispatched out to the robot once every loop cycle.

The reason we are holding down the key is that we otherwise cannot stop an operation that is in progress. Especially in manual control, we don't know the distance, and the robot does not have a method for just starting a motor indefinitely. We thought of being able to do this by loading ACL functions onto the robot to handle this, but it ended up being out of scope for this project.

Ichor

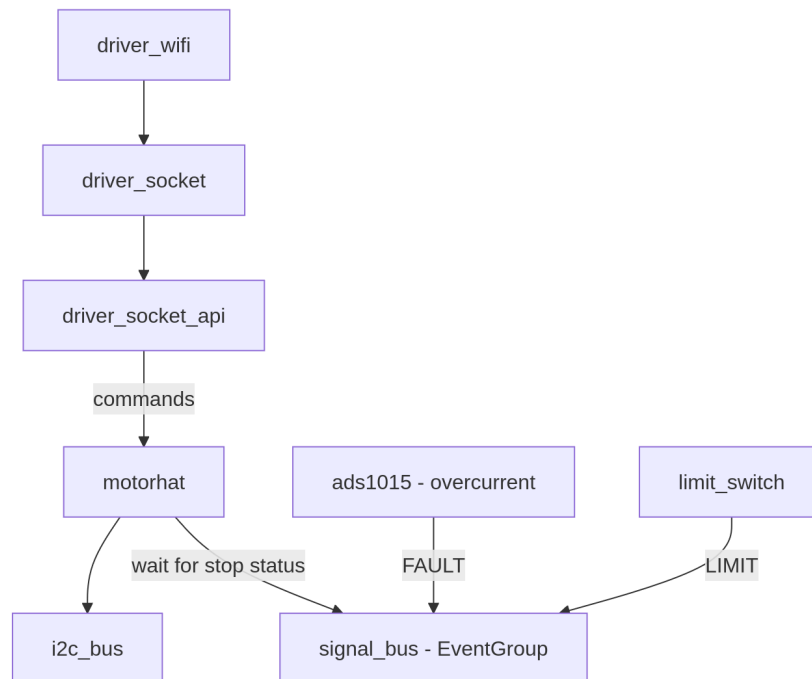
Ichor is the section that was once associated with the ER-4PC. This was the hardware/driver implementation for the Pi. We decided against this in favor of the ESP32 due to the availability of the PCNT (Pulse Count) module and additional GPIO pins. We also had the benefit of not having to run on a Linux OS.

Overall, we did not make significant changes to the architecture of the operator application. This was especially true in the latter half of the project when we were moving closer to deprecating this project in favor of moving everything to the ESP32 platform, including the ACL/serial functionality. We did make some drastic maintainability changes by moving to a modern version of C++. This application was, and still partly uses, outdated C++ practices, many of which you would find in C. Of note, there is an `S_List`, which is a custom linked list, as well as a custom logger, which, although they do work, have well-maintained solutions baked into C++.

ESP Driver

ESP Driver is an embedded application designed to run on an ESP32. This project started as the requirement for more GPIO pins and faster processing, without the limitations of an OS, which became apparent after testing the limits of the Raspberry Pi. Due to the speed and lack of pins on the Pi 4B, we moved to building on top of an ESP32. This project is an [ESP IDF](#) project. ESP-IDF is the official framework built by Espressif. We decided towards this rather than the Arduino framework due to its expanded ability to configure the chip, as well as the added debugging capabilities.

This project is organized into many different components, which are meant to fully replace the Operator application for the ER-4PC. The goal in the future would be to move the serial/ACL logic from Operator into this application and fully deprecate Operator.



All requests come from the WiFi driver of the ESP32, which get received from a TCP socket. The `driver_socket_api` runs the command decoding and processing, which sends data to the `motorhat` (the hardware that drives the motors). There are different EventGroups where the `motorhat` listens on to determine if the motors need to be stopped. This, in the current implementation, is by a limit switch or overcurrent, from the ADS1015. The I2C Bus is a wrapper class to hold a single bus.

This component-based architecture, designed similarly to classes, allows multiple uses of each class without changing the code within. This is possible due to `handle_t` types and

`config_t` types that are managed externally, which allow each creation to have its own state. This also allows the code to be separate and much simpler to comprehend.

This framework is heavily dependent on [FreeRTOS](#). The ESP32 has 2 cores on board, and uses the RTOS to predictably schedule tasks on the CPU. EventGroups, vTaskDelay, Queues, and much more are a part of FreeRTOS.

Menuconfig

The configuration for this project is contained within the Menuconfig/sdkconfig. The sdkconfig and sdkconfig.projbuild files determine configuration parameters, and are accessed using `CONFIG_<the name of the parameter>` within the application. This allows users to set up parameters such as WiFi passwords, ports, I2C addresses, and such without hardcoding them into the application.

I2C Bus

I2C Bus just acts as a wrapper to the ESP IDF I2C Bus. It specifically wraps the [i2c_master_bus_handle_t](#) type. It is used to share the bus between multiple components. It does not wrap the methods; it allows the bus to be fetched, and does not do any verification. It trusts the components using this data are using them correctly.

Because both the PCA9685 (the chip on the motorhat that controls the motor drivers) and the ADS1015 are on the same bus, we needed a wrapper class to keep shared buses, as the driver does not manage this for you.

MotorHAT

There are 2 sections of this: the motorhat section and the PCA9685 section. On the Adafruit MotorHAT, the chip that is connected to the I2C pins is the PCA9685. This chip controls multiple PWM channels and controls the channels directly. The PCA9685 is connected to 2 TB6612FNG motor drivers. Due to the motorHAT not changing, the mapping of PCA channels to pins on the TB6612 are statically mapped and hardcoded within the driver. This should not be used directly. The motorhat section wraps the PCA9685 to the proper channels and allows you to set the speed of the motors directly. This is the public API and is the one that is used in the rest of the source.

The reason we chose to separate the 2 modules was for readability and simplicity. The PCA9685 driver should control only the PCA9685, no matter what else is connected. A higher-level abstraction, the MotorHAT was written to allow for simpler modification and understandability.

ADS1015

The ADS1015 module is an I2C driver for the Texas Instruments ADS1015. Currently, it is designed to detect overcurrent. Due to the functionality of the ADC, we are rapidly swapping the Multiplexer after every conversion in order to use both channels of the ADC. Ideally, we would have 7 single-channel ADCs rather than 8 dual channels with a multiplexer, as in this case, it

hurts more than it helps. The ADC's have a built-in differential on them, but we cannot use it due to this limitation.

Another hurdle we realized is that motor current is not flat, nor is the overcurrent threshold. The idea was to use a function to set the threshold based on actual motor speed determined from the encoder, but that fell out of scope for this project.

Driver Wifi/Driver Socket

The Driver Wifi module sets up the WiFi chip of the ESP32 and connects to the configured network. It is limited up to WPA3 Personal; it cannot, at the moment, connect to enterprise networks. Since we were using a local network during testing anyway, we determined this to be an acceptable limitation.

Driver Socket sets up a TCP socket using the lwIP Socket API (the internet protocol on the ESP32). There are 2 sections, the `driver_socket` and the `driver_socket_api`. The `driver_socket` just sets up a socket and has a loop for accepting connections and commands. It passes it to the `driver_socket_api`, where the processing is held. This is compliant with the ICD api discussed in the Operator section. Commands are sent to the MotorHAT module to be able to run the motors.

This socket was split into 2 sections primarily for readability; it served no functional ability in this context, as they were within the same ESP IDF component.

Encoder

Encoder is the implementation of the PCNT (Pulse Count) module of the ESP IDF. This module is designed to work with quadrature rotary encoders, which we were dealing with. It can count at over 40 MHz, which is the main reason we swapped to the ESP32 in the first place. The ESP32 has 8 channels. The counters are 16 bit, though there is another field for overflow that allows us to count significantly higher, as the base has over 32,000 positions for the encoder. The encoder in its current implementation does not interface yet, as we have not fully implemented closed-loop control, due to scope limitations. In the future, the goal of this would be to help normalize speed, as some motors are faster than others, especially ones that have gravity as an effect, as well as define end stops for the motors.

We realized that the limit switches for the robots are in the center, meaning they are used for homing, and that is it. The encoders would then be calibrated to know when the ends are after homing, which would be used as an ad hoc limit switch at that point.

Limit Switch

This is a simple module that triggers an EventGroup bit when a limit switch gets hit. This allows it to be used in data for homing. In the past, we were using it to stop the robot, but as mentioned in the previous section, the limit switches would have to be used for homing only, as they are in the center of each axis. In its current state, it is being used for a basic homing procedure, though

current sensing needs to be complete before full homing can be used, as if it runs into the other end, it needs to know when to stop, and try another direction.

Overall, the split component architecture of this application makes it maintainable and allows the code to be very reusable. In our setup, we have 7 encoders, which we are able to use the same module multiple times with no duplicate code. We have also implemented some on-device testing with the Unity unit testing framework, which allows us to test specific functionality without modifying the entire main functionality of the application. The goal in the future would be to abstract motor control away from the motorhat to allow ACL to be ported and used to control the ER-V through serial as well.

Hardware

The hardware in this project for the ER-4PC control evolved significantly over the course of the project. Due to the lack of a controller with that robot, we had to design a custom DC motor driver that would support the robot. A Raspberry Pi 4 was used in the previous iteration, and we had started on it, though toward the end of the project, we realized that the speed of encoders, which is around 2-3kHz, was not going to be fast enough, especially through an IO expander, which we were forced to use due to a lack of GPIO pins on the Pi.

Our current hardware layout is defined in hardware schematics developed in KiCad. The overall layout is an Adafruit DC MotorHAT, an ESP32, 4 ADS1015 Analog to Digital converters, 4 logic level shifters, some shunt resistors to measure current, and other various components documented in the schematic. The motorhat was used to directly drive the motors, and although their current was limited for what the motor was capable of driving, it was lower than the measured max current under no load, which, for operating a camera, was sufficient for our use case. The ESP32 was used as the controller, running ESP Driver, being controlled over the internet by Commander. The ADS1015s were reading current by passing the motor signal through shunt resistors. This, in turn, we read the voltage, to which we can calculate the current through the motor. The shifters were used to convert the logic level voltage from the encoders ~4V to 3.3V logic level.

Our future goals would be to convert this to a PCB. Currently, if you draw too much current, you can easily burn a driver on the MotorHAT, as its peak current is around 3.2A. A PCB, especially with stores such as JLBPCB would allow you to easily get it manufactured and assembled. Considering the complexity of this setup, this would be the way to get the best use out of these motors without the risk of burning out the drivers.

Process and Product Metrics

Measurement Strategy

The project utilized a dual-metric approach, tracking both process and product performance. Process metrics included Average PR Confidence, hardware component failure rates (kill count), and total working hours. Product performance was measured by capturing the input and output speeds (FPS) of the object detection model within the Commander TUI.

Quantitative Results

Analysis of the data revealed an Average PR confidence of 4.27/5, supported by over 1000 total team working hours (averaging 8 hours per person per week). However, the technical complexity resulted in a high hardware kill count, including one Raspberry Pi, three ADS 1015 modules, one ESP32, 1.5 motor hats, and a multimeter.

Performance Analysis

The high PR confidence scores suggest that the internal code review process was effective and maintained high quality. While the team showed significant dedication through consistent working hours, the hardware failure rate highlights the volatility and expense of the prototyping phase. A strategic trade-off was made to prioritize functional features over the integration of performance metrics into the GUI, resulting in limited visibility of real-time product performance for the end-user.

- **Went Well:** The high PR confidence and dedication (working hours) point to a successful internal development process and good code quality management.
- **Went Poorly:** The extensive hardware kill count highlights major setbacks and component failures during development. Additionally, the decision to prioritize other features over displaying the software functionality metrics in the GUI suggests a trade-off was made, resulting in incomplete visibility of product performance.

Product State at Time of Delivery

State of Product and Missing Features

1. ER-4PC with ESP Driver
 - a. Controllable for only 2 axes due to hardware restrictions
 - b. Uses ESP driver code ported over from Operator
 - c. Able to Home using 2 of the axes in control
 - d. Able to read current encoder position
 - e. No ability to sense end of its range of motion
2. ER-V with Operator
 - a. Controllable using polar coordinates

- b. No advanced control scheme
 - c. Lack of health monitor and self-resilience
- 3. Commander
 - a. Detects humans from frames pulled from Camera-streamer
 - b. Able to pull frames from multiple robots simultaneously
 - c. Able to control the robot in two axes (up-down, left-right)
 - d. Latency problems and heavily buffered frame
 - e. No audio recordings support
 - f. Timestamps collected but not being utilized
 - g. Lack of health monitor and self-resilience, specifically with the detection process.
It is not restarted if it crashes

Unplanned Implemented Features

- 1. ESP Driver
 - a. The entirety of this software was unplanned due to sudden realization of hardware limitations
 - b. Hardware is still limited, but has improved with realtime capabilities
- 2. Commander
 - a. PyVirtualCam and MediaMTX/ffmpeg Streamer
 - i. This was added as a nice to have and a proof of architectural choice
 - ii. Adding this proved that the section of application is indeed extendable
 - iii. MediaMTX can only be used in headless mode while PyVirtualCam can only be used with the Tkinter or Pyside6 GUI
 - b. Multi-interface
 - i. PySide6 and Textual UI was added to see pros and cons of each interface libraries
 - ii. Pyside6, while still having performance degradation, is much more capable of displaying video streams
 - iii. Textual UI was introduced to allow the usage of commander in headless mode. Currently, the best performing interface with the con of missing video stream

Discrepancy

The ESP driver was a result of uncontrollable hardware limitations and lack of knowledge in electrical. In terms of discrepancy, we promised for a fully functional ER-4PC, but the work in porting over to ESP caused major delays to this promise.

The Commander supports the majority of what was promised, and started to look into non-essential features to show progress and improvements. However, there are some concerns with the Commander's future as it may look like bloated software with the amount of capabilities it can support.

Project Reflection

Right

1. Good team dynamic
 - a. Great team synergy and productive meetings where everyone participates and shares ideas.
 - b. Psychological safety was a key factor in getting the team to be comfortable sharing ideas.
2. Continuous development and releases
 - a. Aside from major setbacks from hardware, we were delivering constant features and improvements to the entire stack, utilizing all of the members to work simultaneously on separate parts of the system.
3. Use of AI to fill in the knowledge gaps and reduce workloads
 - a. Indexed our repos with DeepWiki to create AI documentation that allows us to ask questions to the AI about our project. This was utilized with future teams in mind.

Wrong

1. Project management lacked continuous practice
 - a. The major setback for this project was the fact that we had to switch project methodology for the second semester. Instead, the final process set shape naturally as we worked on the project. However, once the methodology was realized we still did not focus deeply on actual practice and instead focused heavily on feature delivery. Which from the transparency aspect of the project could cause this project to look chaotic and unmanaged with inconsistent updates on kanban board and lack of documentation practice.
2. Lack of metrics
 - a. Partially caused by a lack of software instrumentation to collect product metrics. We instead focused on features rather than the collection of metrics.
3. Inconsistent updates on documentation
 - a. While development started on this project, the features are the only considerations and documentation was an afterthought which is a bad practice considering this project is an open source project because it would make it harder for another group or new members to work on the project.
4. Proper and detailed requirement gathering for Commander
 - a. While not a huge problem, we lacked clearly defined requirements.
5. Lack of hardware knowledge
 - a. We did not have professional knowledge designing hardware, and did not consult others earlier. Consulting others with greater knowledge would cause us to save money by not purchasing unnecessary components.

Proposed Changes

1. Clearly defined document of project practice
 - a. This document should clearly define activities that are required by the project members to execute for every meeting, demo, and even during development.
 - b. An essential step to keeping this document useful is to review it for every retrospective or milestone to propose changes and keep members in check of good practices.
2. Collect more metrics and automate where possible
 - a. Our group did not do this very well and focused on features over development practices. The group should have gathered as much information as possible about the project at the start of the year, but instead jumped into delivering features right away.
 - b. For the future groups, we recommend that they start off by defining clear metrics that are automatically collected and should require no effort to review.
 - c. Additionally, we lacked technical performance metrics and yet assumed delivered features to have improved them based on theory and assumptions. This is a dangerous practice and should be avoided where possible. Commander application saw this at some point during the development and lost actual insight on whether the improvements were real improvements. For this, we recommend a simple benchmarking tool built into the Python standard library, like [timeit](#). This shouldn't require knowledge of the source code and just measures how fast or often a function is being called. Another alternative tool for profiling is scalene; however it has known issues with PyTorch and should be avoided until those are fixed.
3. Hardware knowledge gap
 - a. While an uncontrollable situation, we could have reached out for resources that were always readily available to fill in the missing knowledge.
 - b. We did request for a hardware background developer for the next semester, but getting simple consulting from a few hardware students could have easily mitigated the risk of hardware limitations at an early stage.

References

1. https://deepwiki.com/talos-rit/project_documentation
2. https://github.com/talos-rit/project_documentation